



RELATIONAL DATABASE DESIGN

Module 3

BCSE302L - Database Systems

Introduction

- The goal of a relational-database design is to generate a set of relation schemas that allows **us to store information without unnecessary redundancy**, yet also allows us to **retrieve information easily**.
- One approach is to design schemas that are in an appropriate normal form.
- To determine whether a relation schema is in one of the desirable normal forms, we need additional information about the real-world enterprise that we are modeling with the database.

Normalization

- Normalization is the **process of organizing data** in a database. This includes creating tables and establishing relationships between those tables according to rules designed both to **protect the data and to make the database more flexible by eliminating redundancy and inconsistent dependency.**

Normalization

- A large database defined as a single relation may result in data duplication. This repetition of data may result in:
 - Making **relations very large**.
 - It **isn't easy to maintain and update data** as it would involve searching many records in relation.
 - **Wastage and poor utilization of disk space and resources.**
 - The likelihood of **errors and inconsistencies increases**.

Normalization

- So to handle these problems, we should **analyze and decompose the relations** with redundant data into smaller, simpler, and well-structured relations that satisfy desirable properties.
- Normalization is **a process of decomposing the relations into relations with fewer attributes.**

What is Normalization?

- Normalization is the **process of organizing the data** in the database.
- Normalization is **used to minimize the redundancy** from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization **divides the larger table into smaller and links them using relationships.**
- The normal form is **used to reduce redundancy** from the database table.

Why do we need Normalization?

- The main reason for normalizing the relations is **removing data modification anomalies**.
- **Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other problems** as the database grows.
- Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

Data modification anomalies

- Data modification anomalies can be categorized into three types:
- **Insertion Anomaly:** Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.
- **Deletion Anomaly:** The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.
- **Updation Anomaly:** The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

Types of Normal Forms

	1NF	2NF	3NF	4NF	5NF
Decomposition of Relation	R	R₁₁ R₁₂	R₂₁ R₂₂ R₂₃	R₃₁ R₃₂ R₃₃ R₃₄	R₄₁ R₄₂ R₄₃ R₄₄ R₄₅
Conditions	Eliminate Repeating Groups	Eliminate Partial Functional Dependency	Eliminate Transitive Dependency	Eliminate Multi-values Dependency	Eliminate Join Dependency

Types of Normal Forms

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transitive dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.

Advantages of Normalization

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.
- Much more flexible database design.
- Enforces the concept of relational integrity.

Disadvantages of Normalization

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems.

First Normal Form

- Domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
 - Set of names, composite attributes
- Identification numbers like CS101 that can be broken up into parts
- A relational schema R is in **first normal form** if the domains of all attributes of R are atomic
- **Non-atomic values complicate storage and encourage redundant (repeated) storage of data**
 - E.g. Set of accounts stored with each customer, and set of owners stored with each account

First Normal Form

- Atomicity is actually **a property of how the elements of the domain are used.**
 - E.g. Strings would normally be considered indivisible
 - Suppose that students are given roll numbers which are strings of the form CS0012 or EE1127
 - If the first two characters are extracted to find the department, the domain of roll numbers is not atomic.
 - Doing so is a bad idea: leads to encoding of information in application program rather than in the database.

First Normal Form

- Check Relation EMPLOYEE is in 1NF.

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385, 9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389, 8589830302	Punjab

First Normal Form

- Relation EMPLOYEE is not in 1NF because of multi-valued attribute EMP_PHONE.
- The decomposition of the EMPLOYEE table into 1NF has been shown below:

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
14	John	7272826385	UP
14	John	9064738238	UP
20	Harry	8574783832	Bihar
12	Sam	7390372389	Punjab
12	Sam	8589830302	Punjab

Pitfalls in Relational Database Design

- **Relational database design requires that we find a “good” collection of relation schemas. A bad design may lead to**
 - Repetition of Information.
 - Inability to represent certain information.
- **Design Goals:**
 - Avoid redundant data
 - Ensure that relationships among attributes are represented
 - Facilitate the checking of updates for violation of database integrity constraints.

Pitfalls in Relational Database Design

- Consider the relation schema:

*Lending-schema = (branch-name, branch-city, assets,
customer-name, loan-number, amount)*

<i>branch-name</i>	<i>branch-city</i>	<i>assets</i>	<i>customer-name</i>	<i>loan-number</i>	<i>amount</i>
Downtown	Brooklyn	9000000	Jones	L-17	1000
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Downtown	Brooklyn	9000000	Jackson	L-14	1500

Pitfalls in Relational Database Design

- **Redundancy:**

- Data for branch-name, branch-city, assets are repeated for each loan that a branch makes
- Wastes space
- Complicates updating, introducing possibility of inconsistency of assets value

- **Null values**

- Cannot store information about a branch if no loans exist
- Can use null values, but they are difficult to handle.

Decomposition

- Decompose the relation schema Lending-schema into:

Branch-schema = (branch-name, branch-city, assets)

Loan-info-schema = (customer-name, loan-number, branch-name, amount)

- All attributes of an original schema (R) must appear in the decomposition (R_1, R_2) :

$$R = R_1 \cup R_2$$

- Lossless-join decomposition.
For all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

Example of Non Lossless-Join Decomposition

- Decomposition of $R = (A, B)$ into $R_1 = (A)$, $R_2 = (B)$

A	B
α	1
α	2
β	1

r



A
α
β

$\Pi_A(r)$

B
1
2

$\Pi_B(r)$



A	B
α	1
α	2
β	1
β	2

$\Pi_A(r) \bowtie \Pi_B(r)$

Goal

- Decide whether a particular relation R is in “good” form.
- In the case that a relation R is not in “good” form, decompose it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - each relation is in good form
 - the decomposition is a lossless-join decomposition
- Our theory is based on:
 - functional dependencies
 - multivalued dependencies

Functional Dependencies

- **Constraints on the set of legal relations.**
- Require that the **value for a certain set of attributes determines uniquely the value for another set of attributes.**
- A functional dependency is a generalization of the notion of a **key**.

Functional Dependencies

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$
- The functional dependency $\alpha \rightarrow \beta$ holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α , they also agree on the attributes β .

$$\text{i.e., } t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A, B)$ with the following instance of r .

1	4
1	5
3	7

- On this instance, $A \rightarrow B$ does NOT hold, but $B \rightarrow A$ does hold.

Functional Dependencies

- K is a **superkey** for relation schema R if and only if $K \rightarrow R$
- K is a **candidate key** for R if and only if $K \rightarrow R$, and for no $\alpha \subset K, \alpha \rightarrow R$
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:
Loan-info-schema = (customer-name, loan-number, branch-name, amount)
- We expect this set of functional dependencies to hold:
loan-number $\rightarrow$ amount, loan-number $\rightarrow$ branch-name
- but would not expect the following to hold: loan-number $\rightarrow$ customer-name

Functional Dependencies

- We use functional dependencies to:
 - test relations to see if they are legal under a given set of functional dependencies.
 - If a relation r is legal under a set F of functional dependencies, we say that r **satisfies** F .
 - specify constraints on the set of legal relations
 - We say that F **holds on** R if all legal relations on R satisfy the set of functional dependencies F .

Functional Dependencies

- **Trivial functional dependency**
 - $A \rightarrow B$ has trivial functional dependency if B is a subset of A .
 - The following dependencies are also trivial like: $A \rightarrow A, B \rightarrow B$
- **Example:** Consider a table with two columns `Employee_Id` and `Employee_Name`. $\text{Employee_id}, \text{Employee_Name} \rightarrow \text{Employee_Id}$ is a trivial functional dependency as
 - Employee_Id is a subset of $\{\text{Employee_Id}, \text{Employee_Name}\}$.
 - Also, $\text{Employee_Id} \rightarrow \text{Employee_Id}$ and $\text{Employee_Name} \rightarrow \text{Employee_Name}$ are trivial dependencies too.

Functional Dependencies

- **Non-trivial functional dependency**
 - $A \rightarrow B$ has trivial functional dependency if B is not a subset of A .
 - When A intersection B is NULL, then $A \rightarrow B$ is called as complete non-trivial.
- **Example:**
 - $Employee_id \rightarrow salary$
 - $Employee_Name \rightarrow address$

Functional Dependencies

- List all functional dependencies for given relation.

Customer_name	Manager_name	Branch
Karthik	Ramya	SBI
Karthik	Ramya	AXIS
Pricilla	Ramya	SBI
Pricilla	Ramya	CUB

- Customer_name → Manager_name
- Branch → Manager_name
- Logically implied Customer_name, Branch → Manager_name.

Closure of a set of FDs

- Closure of F by F^+ found by applying Armstrong's Axioms:
 - if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ **(reflexivity)**
 - if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$ **(augmentation)**
 - if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ **(transitivity)**
- These rules are
 - **sound** (generate only functional dependencies that actually hold) and
 - **complete** (generate all functional dependencies that hold).

Procedure for Computing F^+ Example

- We can further simplify manual computation of F^+ by using the following additional rules.
 - If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta \gamma$ holds (union)
 - If $\alpha \rightarrow \beta \gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds (decomposition)
 - If $\alpha \rightarrow \beta$ holds and $\gamma \beta \rightarrow \delta$ holds, then $\alpha \gamma \rightarrow \delta$ holds (pseudotransitivity)
- The above rules can be inferred from Armstrong's axioms.

Procedure for Computing F^+ Example

- $F^+ = F$
 repeat
 for each functional dependency f in F^+
 apply reflexivity and augmentation rules on f
 add the resulting functional dependencies to F^+
 for each pair of functional dependencies f_1 and f_2 in F^+
 if f_1 and f_2 can be combined using transitivity
 then add the resulting functional
 dependency to F^+
 until F^+ does not change any further

Closure of a set of FDs - Example

$$R = (A, B, C, G, H, I),$$

$$F = \{ A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H \}$$

- To Compute F^+
 - $A \rightarrow H$ - by **transitivity** from $A \rightarrow B$ and $B \rightarrow H$
 - $AG \rightarrow I$ - by **augmenting** $A \rightarrow C$ with G , to get $AG \rightarrow CG$ and then **transitivity** with $CG \rightarrow I$
 - $CG \rightarrow HI$ - from $CG \rightarrow H$ and $CG \rightarrow I$: "**union rule**" can be inferred from definition of functional dependencies, or **Augmentation** of $CG \rightarrow I$ to infer $CG \rightarrow CGI$, **augmentation** of $CG \rightarrow H$ to infer $CGI \rightarrow HI$, and then **transitivity**

Closure of Attribute Sets

- Given a set of attributes a , define the closure of a under F (denoted by a^+) as the set of attributes that are functionally determined by a under F :

$$a \rightarrow \beta \text{ is in } F^+ \Leftrightarrow \beta \subseteq a^+$$

Closure of Attribute Sets

- Algorithm to compute a^+ , the closure of a under F
result := a ;
 while (changes to result) **do**
 for each $\beta \rightarrow \gamma$ in F **do**
 Begin
 if $\beta \subseteq \text{result}$ then result := result $\cup \gamma$
 end

Closure of Attribute Set- Example

$$R = (A, B, C, G, H, I),$$

$$F = \{ A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H \}$$

- To Compute AG^+
 - result = AG
 - result = ABCG $(A \rightarrow C \text{ and } A \rightarrow B)$
 - result = ABCGH $(CG \rightarrow H \text{ and } CG \subseteq AGBC)$
 - result = ABCGHI $(CG \rightarrow I \text{ and } CG \subseteq AGBCH)$

Use of Attribute Closure- Example

$$R = (A, B, C, G, H, I),$$

$$F = \{ A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H \}$$

- Is AG a candidate key?
 - Is AG a super key? -Does $AG \rightarrow R$? $==$ Is $(AG)^+ \supseteq R$
 - Is any subset of AG a superkey?
 - Does $A \rightarrow R$? $==$ Is $(A)^+ \supseteq R$
 - Does $G \rightarrow R$? $==$ Is $(G)^+ \supseteq R$

Closure of Attribute Sets

- **Testing for superkey:**
 - To test if α is a superkey, we compute α^+ , and check if α^+ contains all attributes of R .
- **Testing functional dependencies**
 - To check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if $\beta \subseteq \alpha^+$.
 - That is, we compute α^+ by using attribute closure, and then check if it contains β .
 - Is a simple and cheap test, and very useful

Closure of Attribute Sets

- **Computing closure of F**
- For each $\gamma \subseteq R$, we find the closure γ^+ , and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.

Canonical Cover

- Sets of functional dependencies may have redundant dependencies that can be inferred from the others
- Eg: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
- Parts of a functional dependency may be redundant
- E.g. on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
- E.g. on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

Canonical Cover – Extraneous Attribute

- Note: implication in the opposite direction is trivial in each of the cases above, since a “stronger” functional dependency always implies a weaker one
- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
 - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$ (i.e. the result of dropping B from $AB \rightarrow C$).
- Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$
 - C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C

Testing if an Attribute is Extraneous

- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F .
- To test if attribute $A \in \alpha$ **is extraneous in α**
 - Compute $\gamma = \{\alpha\} - A$ using the dependencies in F
 - Check γ^+ contains all attribute of β then A is extraneous
- To test if attribute $A \in \beta$ **is extraneous in β**
 - Compute $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$, if $\alpha \rightarrow A$ can be inferred with F'
 - Compute α^+ under F' and check α^+ contains A ; if it does, A is extraneous

Canonical Cover

- A canonical cover for F is a set of dependencies F_c such that
 - F logically implies all dependencies in F_c , and
 - F_c logically implies all dependencies in F , and
 - No functional dependency in F_c contains an extraneous attribute, and
 - Each left side of functional dependency in F_c is unique.

Canonical Cover

- To compute a canonical cover for F :

repeat

Use the union rule to replace any dependencies in F

$\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1\beta_2$

Find a functional dependency $\alpha \rightarrow \beta$ with an extraneous attribute either in α or in β

If an extraneous attribute is found, delete it from

$\alpha \rightarrow \beta$

until F does not change

- Note: Union rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied

Canonical Cover - Example

- $R = (A, B, C)$
 $F = \{A \rightarrow BC$
 $B \rightarrow C$
 $A \rightarrow B$
 $AB \rightarrow C\}$

- **Step 1:**

- Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
 - Set is now $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$

Canonical Cover

- $R = (A, B, C)$
 $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$
- $F_c = \{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- **Step 2:**
- **A is extraneous in $AB \rightarrow C$**
 - Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies
 - Yes: in fact, $B \rightarrow C$ is already present!
 - Set is now $\{A \rightarrow BC, B \rightarrow C\}$

Canonical Cover

- $R = (A, B, C)$
 $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$
- $F_c = \{A \rightarrow BC, B \rightarrow C, \}$
- **Step 3:**
- **C is extraneous in $A \rightarrow BC$**
 - Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and the other dependencies
 - Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.
 - Can use attribute closure of A in more complex cases

Canonical Cover

- $R = (A, B, C)$
 $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$
- **$F_c = \{A \rightarrow C, B \rightarrow C, \}$**

Goals of Normalization

- Decide whether a particular relation R is in “good” form.
- In the case that a relation R is not in “good” form, decompose it into a set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - each relation is in good form
 - the decomposition is a lossless-join decomposition
- Our theory is based on:
 - functional dependencies
 - multivalued dependencies

Decomposition

- Decompose the relation schema Lending-schema into:
 Branch-schema = (branch-name, branch-city, assets)
 Loan-info-schema = (customer-name, loan-number, branch-name, amount)
- All attributes of an original schema (R) must appear in the decomposition (R_1, R_2): $R = R_1 \cup R_2$
- Lossless-join decomposition: For all possible relations r on schema R : $r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$
- A decomposition of R into R_1 and R_2 is lossless join if and only if at least one of the following dependencies is in F^+ :

$$R_1 \cap R_2 \rightarrow R_1$$
$$R_1 \cap R_2 \rightarrow R_2$$

Example of Lossy-Join Decomposition

- Lossy-join decompositions result in information loss.
- Example: Decomposition of $R = (A, B)$, $R_1 = (A)$, $R_2 = (B)$

A	B
α	1
α	2
β	1

r

A	B
α	1
β	2

$\Pi_A(r)$ $\Pi_B(r)$

A	B
α	1
α	2
β	1
β	2

$\Pi_A(r) \bowtie \Pi_B(r)$

Normalization Using Functional Dependency

- When we decompose a relation schema R with a set of functional dependencies F into $R_1, R_2, \dots, R_n$ we want
 - **Lossless-join decomposition**: Otherwise decomposition would result in information loss.
 - **No redundancy**: The relations R_i preferably should be in either Boyce-Codd Normal Form or Third Normal Form.
 - **Dependency preservation**: Let F_i be the set of dependencies F^+ that include only attributes in R_i .
 - Preferably the decomposition should be dependency preserving, that is, $(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$
 - Otherwise, checking updates for violation of functional dependencies may require computing joins, which is expensive

Example

- $R = (A, B, C), F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
$$R_1 = (A, B), \quad R_2 = (B, C)$$

- Lossless-join decomposition:

$$R_1 \cap R_2 = \{B\} \text{ and } B \rightarrow BC$$

- Dependency preserving

$$R_1 = (A, B), \quad R_2 = (A, C)$$

- Lossless-join decomposition:

$$R_1 \cap R_2 = \{A\} \text{ and } A \rightarrow AB$$

- Not dependency preserving
(cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

Testing for Dependency Preservation

```
compute  $F^+$ ;  
for each schema  $R_i$  in  $D$  do  
    begin  
         $F_i :=$  the restriction of  $F^+$  to  $R_i$ ;  
    end  
 $F^+ := \emptyset$   
for each restriction  $F_i$  do  
    begin  
         $F' = F' \cup F_i$   
    end  
compute  $F'^+$ ;  
if ( $F'^+ = F^+$ ) then return (true) else return (false);
```

Testing for Dependency Preservation

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \dots, R_n$ we apply the following simplified test (with attribute closure done w.r.t. F)
- $result = \alpha$
while (changes to result) **do**
 for each R_i in the decomposition
 $t = (result \cap R_i) + \cap R_i$
 $result = result \cup t$

Testing for Dependency Preservation

- If result contains all attributes in β , then the functional dependency $\alpha \rightarrow \beta$ is preserved.
- We apply the test on all dependencies in F to check if a decomposition is dependency preserving
- This procedure takes polynomial time, instead of the exponential time required to compute

$$F^+ \text{ and } F' = (F_1 \cup F_2 \cup \dots \cup F_n)$$

- **Dependency Preserving:** $F^+ = F'^+$

Second Normal Form

- A **candidate key**, or simply a **key**, of a relational database is a minimal superkey.
- It is any set of columns that have a unique combination of values in each row (which makes it a superkey), with the additional constraint that removing any column could produce duplicate combinations of values (which makes it a minimal superkey).
- Candidate keys are sometimes called **primary keys, secondary keys** or **alternate keys**
 - The columns in a candidate key are called **prime attributes**
 - The column that does not occur in any candidate key is called a **non-prime attribute**.

Second Normal Form

- **Functionally Dependent**

- A relation R and sets of attributes $X, Y \subseteq R$, X is said to functionally determine Y (written $X \rightarrow Y$) if and only if each X value in R is associated with precisely one Y value in R ; R is then said to satisfy the functional dependency $X \rightarrow Y$.

Second Normal Form

- **Second normal form (2NF)** is a normal form used in database normalization.
- 2NF was originally defined by E. F. Codd in 1971

Second Normal Form

- A relation is in the 'second normal form if it fulfills the following two requirements:
 - It is in **1NF**.
 - No non-prime attribute that is functionally dependent on any proper subset of any candidate key of the relation. A **non-prime attribute** of a relation is an attribute that is not a part of any candidate key of the relation.

Or

- It is in 1NF
- It should not have any Partial Dependencies

Partial Dependency

- Partial dependency occurs when one primary key determines some other attribute/attributes. On the other hand, transitive dependency occurs when some non-key attribute determines some other attribute.
- The FD (functional dependency) $A \rightarrow B$ happens to be a partial dependency if B is functionally dependent on A, and also B can be determined by any other proper subset of A.
- For instance, we have a relationship like **$MO \rightarrow N$, $M \rightarrow P$, and $P \rightarrow N$** . In this case, M is alone capable of determining N. It means that N is dependent partially on MO.

Partial Dependency

- A partial dependency **would occur whenever a non-prime attribute depends functionally on a part of the given candidate key**. The 2NF (Second Normal Form) eliminates partial dependency.
- Example

<u>Emp_ID</u>	<u>Task_No</u>	Emp_name	Task_name
1	003	John	Programmer
2	004	Rhodes	Tester

Partial Dependency

- Given Relation violates 2NF so we decompose the tables to remove Partial Dependency along with the violation on the 2NF.

<u>Emp_ID</u>	Task_No	Emp_name
1	003	John
2	004	Rhodes

<u>Task_No</u>	Task_name
003	Programmer
004	Tester

- Thus Decomposed relations in 2NF.

Second Normal Form

- To make a 1NF relation a 2NF relation,
 - **Remove the functionally dependent attributes in the partial dependencies** of the first normal form relation,
 - Place those **partial dependency dependent attributes in a relation where their corresponding determinant attributes are an entire candidate key**.

Second Normal Form - Example

- The following relation does not satisfy 2NF because:
- {Manufacturer country} is **functionally dependent on** {Manufacturer}.
- {Manufacturer country} is a **non-prime attribute**.
- {Manufacturer} is a **proper subset of the** {Manufacturer, Model} candidate key.

Electric toothbrush models

<u>Manufacturer</u>	<u>Model</u>	Manufacturer country
Forte	X-Prime	Italy
Forte	Ultraclean	Italy
Dent-o-Fresh	EZbrush	USA
Brushmaster	SuperBrush	USA
Kobayashi	ST-60	Japan
Hoch	Toothmaster	Germany
Hoch	X-Prime	Germany

Second Normal Form - Example

- {Manufacturer country} is a **non-prime attribute functionally dependent on a proper subset of a candidate key**, the relation is in violation of 2NF.

Electric toothbrush models

<u>Manufacturer</u>	<u>Model</u>	Manufacturer country
Forte	X-Prime	Italy
Forte	Ultraclean	Italy
Dent-o-Fresh	EZbrush	USA
Brushmaster	SuperBrush	USA
Kobayashi	ST-60	Japan
Hoch	Toothmaster	Germany
Hoch	X-Prime	Germany

Second Normal Form - Example

- To make the design conform to 2NF, it is necessary to have two relations. To create these relations:
 - Remove the functionally dependent attributes in the partial dependencies of the first normal form relation.
 - {Manufacturer country} is the functionally dependent attribute which will be removed.
 - Place those partial dependency dependent attributes (i.e. {Manufacturer country}) in a relation where their corresponding determinant attributes are a candidate key (i.e. {Manufacturer}).

Second Normal Form - Example

- {Manufacturer country} is removed from the original table.
- Partial dependency is put into a new relation where the dependency can exist without being a partial dependency

Electric toothbrush models

<u>Manufacturer</u>	<u>Model</u>
Forte	X-Prime
Forte	Ultraclean
Dent-o-Fresh	EZbrush
Brushmaster	SuperBrush
Kobayashi	ST-60
Hoch	Toothmaster
Hoch	X-Prime

Electric toothbrush manufacturers

<u>Manufacturer</u>	Manufacturer country
Forte	Italy
Dent-o-Fresh	USA
Brushmaster	USA
Kobayashi	Japan
Hoch	Germany

Second Normal Form – Example 2

- Candidate Key – {Account_no, Branch_name}
- Non prime Attribute – {Amount}

<u>Account no</u>	<u>Branch name</u>	Amount
1	SBI	400
1	AXIS	400
2	ICICI	400
3	AXIS	500
3	PNB	500

- It is **not in 2NF**, since {Amount} is dependent on Account_no where {Account_no} $\subseteq$ {Account_no, Branch_name}

Second Normal Form – Example 2

- To make it 2NF we decompose the relation

<u>Account no</u>	Amount
1	400
2	400
3	500

<u>Account no</u>	<u>Branch name</u>
1	SBI
1	AXIS
2	ICICI
3	AXIS
3	PNB

- Reason is, Same account used in different branches, so using amount we cannot find account, branch.
- Using branch and amount we cannot find account.

Third Normal Form

- The third normal form (3NF) is a normal form used in database normalization.
- 3NF was originally defined by E. F. Codd in 1971.

Third Normal Form

- A table is in 3NF if and only if both of the following conditions hold:
 - The relation R (table) is in second normal form (2NF).
 - No non-prime attribute of R is transitively dependent on the primary key.
- or
- $\alpha \rightarrow \beta$ is Trivial Functional dependency
 - α is super key for relation R
 - Each attribute A in $\beta - \alpha$ is contained in a candidate key of R

Transitive Dependency

- Let A, B , and C designate three distinct (but not necessarily disjoint) sets of attributes of a relation. Suppose all three of the following conditions hold:
 - $A \rightarrow B$
 - It is not the case that $B \rightarrow A$
 - $B \rightarrow C$
- Then the functional dependency $A \rightarrow C$ (which follows from 1 and 3 by the axiom of transitivity) is **a transitive dependency**.
- **Transitive Dependency**: A functional dependency formed by two functional dependency indirectly.

Transitive Dependency

- Example

<u>Show ID</u>	<u>Telecast ID</u>	Telecast_Type	CD_cost
1	003	Thriller	50
2	004	Drama	30
3	003	Thriller	20

- $Show_{ID} \rightarrow Telecast_ID, Telecast_ID \rightarrow Telecast_Type$
thus $Show_ID \rightarrow Telecast_Type$ which is transitive dependency.
- Relation is not in 3NF because it includes a TFD.
- To remove this we want to decompose the relations

Transitive Dependency

- To remove TFD, we want to decompose the relations.

<u>Show ID</u>	<u>Telecast ID</u>	Telecast_Type	CD_cost
1	003	Thriller	50
2	004	Drama	30
3	003	Thriller	20

<u>Show ID</u>	<u>Telecast ID</u>	CD_cost
1	003	50
2	004	30
3	003	20

<u>Telecast ID</u>	Telecast_Type
003	Thriller
004	Drama

- Now table is in 3NF.

Transitive Dependency

- Transitive Dependency:

<u>Account no</u>	Customer_name	Customer_street	Customer_city
1	John	Main	Mumbai
2	John	MG Road	Pune

- Customer_name → {Customer_street, Customer_city} but Customer_name, Customer_street, Customer_city are independent of {Customer_name, Account_no}
- An important features of **3NF** that it excludes certain types of transitive dependencies. Invented by E.F. Codd, in 1971

Testing for 3NF

- **Need to check only FDs** in F , need not check all FDs in F^+ .
- **Use attribute closure** to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is **not a superkey**, we have to **verify if each attribute in β is contained in a candidate key** of R
 - Test is rather more expensive, since it involve finding candidate keys.
 - Testing for 3NF has been shown to be NP-hard
 - Decomposition into 3NF can be done in polynomial time.

Testing for 3NF

- Let F_c be a canonical cover for F_i
 $i := 0$;
 for each functional dependency $\alpha \rightarrow \beta$ in F_c **do**
 if none of the schemas $R_j, 1 \leq j \leq i$ contains $\alpha \beta$
 then begin
 $i := i + 1$;
 $R_i := \alpha \beta$
 end
 if none of the schemas $R_j, 1 \leq j \leq i$ contains a candidate key for R
 then begin
 $i := i + 1$;
 $R_i :=$ any candidate key for R ;
 end
 return $(R_1, R_2, \dots, R_i)$

Third Normal Form

- Example

- $R = (J, K, L)$

- $F = \{JK \rightarrow L, L \rightarrow K\}$

- Two candidate keys: JK and JL

- R is in 3NF

- $JK \rightarrow L$ **JK is a superkey**

- $L \rightarrow K$ **K is contained in a candidate key**

Decomposition has (JL) and (LK)

Testing for $JK \rightarrow L$ requires a join

Third Normal Form

- Example

<u>Emp_id</u>	Emp_name	Emp_zip	Emp_state	Emp_city
1	John	101	Mumbai	Pune
2	Jack	123	Kerala	Cochin
3	Shruthi	315	Tamilnadu	Chennai
4	Ramya	412	Uttarpradesh	Noida
5	Sanjay	545	Madhyapradesh	Bhopal

- Super key : *Emp_id, {Emp_id, Emp_name}*
- Candidate key: *Emp_id*
- Non Prime Attributes: *Emp_name, Emp_zip, Emp_state, Emp_city*

Third Normal Form

- Example

<u>Emp_id</u>	Emp_name	Emp_zip	Emp_state	Emp_city
1	John	101	Mumbai	Pune
2	Jack	123	Kerala	Cochin
3	Shruthi	315	Tamilnadu	Chennai
4	Ramya	412	Uttarpradesh	Noida
5	Sanjay	545	Madhyapradesh	Bhopal

- *Emp_state* and *Emp_city* dependent on *Emp_zip*
- *Emp_zip* dependent on *Emp_id*
- NPA *Emp_state* and *Emp_city* transitive dependent on *Emp_id*.
- It violates the 3NF.

Third Normal Form

- So we decompose relation to solve TFD.

<u>Emp_id</u>	Emp_name	Emp_zip
1	John	101
2	Jack	123
3	Shruthi	315
4	Ramya	412
5	Sanjay	545

Emp_zip	Emp_state	Emp_city
101	Mumbai	Pune
123	Kerala	Cochin
315	Tamilnadu	Chennai
412	Uttarpradesh	Noida
545	Madhyapradesh	Bhopal

Third Normal Form

- Example

Stud_id	Stud_name	Stud_state	Stud_country	Stud_age
1	John	Mumbai	India	Pune
2	Jack	Kerala	India	Cochin
3	Shruthi	Tamilnadu	India	Chennai
4	Ramya	Kerala	India	Cochin
5	Sanjay	Tamilnadu	India	Chennai

- FD: Stud_id → Stud_name,
Stud_id → Stud_state,
Stud_state → Stud_country,
Stud_id → Stud_age

Candidate Key: Stud_id

Third Normal Form

- Example

Stud_id	Stud_name	Stud_state	Stud_country	Stud_age
1	John	Mumbai	India	Pune
2	Jack	Kerala	India	Cochin
3	Shruthi	Tamilnadu	India	Chennai
4	Ramya	Kerala	India	Cochin
5	Sanjay	Tamilnadu	India	Chennai

- $\text{Stud_id} \rightarrow \text{Stud_state}$ and $\text{Stud_state} \rightarrow \text{Stud_country}$
- By apply transitive rule: $\text{Stud_id} \rightarrow \text{Stud_country}$. i.e. Stud_country is transitive dependent on Stud_id .
- Violation of 3NF.

Third Normal Form

- We decompose relation to satisfy 3NF

Stud_id	Stud_name	Stud_state	Stud_age
1	John	Mumbai	Pune
2	Jack	Kerala	Cochin
3	Shruthi	Tamilnadu	Chennai
4	Ramya	Kerala	Cochin
5	Sanjay	Tamilnadu	Chennai

Stud_state	Stud_country
Mumbai	India
Kerala	India
Tamilnadu	India
Kerala	India
Tamilnadu	India

Third Normal Form

- Consider relation $R(A,B,C,D,E)$, $F=\{ A \rightarrow B, BD \rightarrow E, B \rightarrow E, E \rightarrow C \}$. Is relation is in 3NF?
- Solution
- **Candidate Key:** A
- **Non prime attribute:** B, C, D, E .
- There is **no partial dependency** so relation R is in 2NF.
- But **there is transitive dependency** $A \rightarrow B$ & $B \rightarrow E$ then $A \rightarrow E$
- Remove TFD by decomposing the relation R

Third Normal Form

- $R(A,B,C,D,E), F=\{ A \rightarrow B, BD \rightarrow E, B \rightarrow E, E \rightarrow C \}$.
- We decompose based on FD
- **Option 1: $R_1(A,B) R_2(B,D,E) R_3(E,C)$**
 - Lossless decomposition
 - R_1 and R_2 are in 2NF and 3NF, but R_2 is not in 2NF since it has partial dependency
 - So decompose R_2 into $R_{21}(B,E)$ and $R_{22}(B,D)$.
 - All the relation R_1, R_{21}, R_{22}, R_3 are without partial dependency and transitive dependency.
- Is Dependency is preserved?

Third Normal Form

- $R(A,B,C,D,E)$, $F = \{ A \rightarrow B, BD \rightarrow E, B \rightarrow E, E \rightarrow C \}$.
- Option 1: **$R_1(A,B)$ $R_{21}(B,E)$ $R_{22}(B,D)$ $R_3(E,C)$**
 - For R_1 : FD $A \rightarrow B$ holds
 - For R_{21} : FD $B \rightarrow E$ holds.
 - For R_{22} : :
 - For R_3 : FD $E \rightarrow C$ holds
- **$F' = \{ A \rightarrow B, B \rightarrow E, E \rightarrow C \}$**
- To check $F^+ = F'^+$ for function dependency preservation.

Third Normal Form

- **$R(A,B,C,D,E)$, $F=\{ A \rightarrow B, BD \rightarrow E, B \rightarrow E, E \rightarrow C \}$.**
- **Compute F^+** under F
 - $A \rightarrow B$ and $B \rightarrow D$ so $A \rightarrow D$
 - $BD \rightarrow E$ and $E \rightarrow C$ so $BD \rightarrow C$
 - **$F^+ = \{A \rightarrow E, BD \rightarrow C, AD \rightarrow E, B \rightarrow C\} \equiv \{A \rightarrow E, B \rightarrow C\}$** since A and B is CK so AD and BD Functional dependencies can be removed.
- **Compute F'^+** using **$F' = \{ A \rightarrow B, B \rightarrow E, E \rightarrow C \}$**
 - $A \rightarrow B$ and $B \rightarrow E$ so $A \rightarrow E$
 - $B \rightarrow E$ and $E \rightarrow C$ so we get $B \rightarrow C$
 - **$F'^+ = \{A \rightarrow E, B \rightarrow C\} \equiv \{A \rightarrow E, AD \rightarrow E, B \rightarrow C, BD \rightarrow C\}$**
- So **$F^+ = F'^+$** hence function dependency is preserved, so all are in **3NF**.

Third Normal Form

- Consider relation $R(A,B,C,D,E)$, $F=\{ A \rightarrow B, BD \rightarrow E, B \rightarrow D, E \rightarrow C \}$. Is relation is in 3NF?
- What are other options for decomposition
- **$R_1(A, B, D, E)$ and $R_2(E, C)$ is 3NF?**

Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:
 - $\alpha \rightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$)
 - α is a **superkey** for R

Boyce-Codd Normal Form

- BCNF (Boyce Codd Normal Form) is the advanced version of 3NF.
- A table is in BCNF if every functional dependency $X \rightarrow Y$,
- X is the super key of the table.
- For BCNF, the table should be in 3NF, and for every FD.
- LHS is super key.

Example

- Consider a relation R with attributes (student, subject, teacher).

Student	Teacher	Subject
John	Das	DBMS
John	Karthik	DS
Shruthi	Das	DBMS
Shruthi	Das	DS

- $F = \{(student, Teacher) \rightarrow subject, (student, subject) \rightarrow Teacher, Teacher \rightarrow subject\}$

Example

Student	Teacher	Subject
John	Das	DBMS
John	Karthik	DS
Shruthi	Das	DBMS
Shruthi	Das	DS

- Candidate keys are (student, teacher) and (student, subject).
- The **above relation is in 3NF** [since there is no transitive dependency].
- A relation R is in BCNF if for every non-trivial FD $X \rightarrow Y$, X must be a key.
- The **above relation is not in BCNF**, because in the FD ($teacher \rightarrow subject$), teacher is not a key. This relation suffers with anomalies
- For example, if ***we try to delete the student Shruthi, we will lose the information that Das teaches DS***. These difficulties are caused by the fact the teacher is determinant but not a candidate key.

Example

Student	Teacher	Subject
John	Das	DBMS
John	Karthik	DS
Shruthi	Das	DBMS
Shruthi	Das	DS

- **Decomposition for BCNF**

- Teacher $\rightarrow$ subject violates BCNF [since teacher is not a candidate key].
- If $X \rightarrow Y$ violates BCNF then divide R into $R1(X, Y)$ and $R2(R - Y)$.
- So R is divided into two relations $R1(Teacher, subject)$ and $R2(student, Teacher)$.

Example

Student	Teacher	Subject
John	Das	DBMS
John	Karthik	DS
Shruthi	Das	DBMS
Shruthi	Das	DS

Table R

Table R1

Teacher	Subject
Das	DBMS
Karthik	DS
Das	DS

Table R2

Student	Teacher
John	Das
John	Karthik
Shruthi	Das
Shruthi	Das

Testing for BCNF

- To check if a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF
 - Compute α^+ (the attribute closure of α), and
 - Verify that it includes all attributes of R , that is, **it is a superkey of R** .
- **Simplified test:** To check if a relation schema R is in BCNF, it suffices to **check only the dependencies in the given set F for violation of BCNF**, rather than checking all dependencies in F^+ .
 - If none of the dependencies in F causes a violation of BCNF, then none of the dependencies in F^+ will cause a violation of BCNF either.

Testing for BCNF

- However, using only F is incorrect when testing a relation in a decomposition of R
 - E.g. Consider $R(A, B, C, D)$, with $F = \{A \rightarrow B, B \rightarrow C\}$
 - Decompose R into $R_1(A, B)$ and $R_2(A, C, D)$
 - Neither of the dependencies in F contain only attributes from (A, C, D) so we might be misled into thinking R_2 satisfies BCNF.
 - In fact, dependency $A \rightarrow C$ in F^+ shows R_2 is not in BCNF.

BCNF Decomposition Algorithm

```
result := {R};  
done := false;  
compute  $F^+$ ;  
while (not done) do  
    if (there is a schema  $R_i$  in result that is not in BCNF)  
        then begin  
            let  $\alpha \rightarrow \beta$  be a nontrivial functional dependency that holds on  $R_i$   
            such that  $\alpha \rightarrow R_i$  is not in  $F^+$ ,  
            and  $\alpha \cap \beta = \emptyset$ ;  
            result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );  
        end  
    else done := true;
```

Note: each R_i is in BCNF, and decomposition is lossless-join.

Example for BCNF Decomposition

- $R = (\text{branch-name}, \text{branch-city}, \text{assets}, \text{customer-name}, \text{loan-number}, \text{amount})$
- $F = \{\text{branch-name} \rightarrow \text{assets} \text{ branch-city}, \text{loan-number} \rightarrow \text{amount} \text{ branch-name}\}$
- $\text{Key} = \{\text{loan-number}, \text{customer-name}\}$
- Decomposition
 - $R_1 = (\text{branch-name}, \text{branch-city}, \text{assets})$
 - $R_2 = (\text{branch-name}, \text{customer-name}, \text{loan-number}, \text{amount})$
 - $R_3 = (\text{branch-name}, \text{loan-number}, \text{amount})$
 - $R_4 = (\text{customer-name}, \text{loan-number})$
- Final decomposition R_1, R_3, R_4

Testing Decomposition for BCNF

- To check if a relation R_i in a decomposition of R is in BCNF,
 - Either test R_i for BCNF with respect to the **restriction** of F to R_i (that is, all FDs in F^+ that contain only attributes from R_i)
 - or use the original set of dependencies F that hold on R , but with the following test:
 - for every set of attributes $\alpha \subseteq R_i$, check that α^+ (the attribute closure of α) either includes no attribute of $R_i - \alpha$, or includes all attributes of R_i .
 - If the condition is violated by some $\square \rightarrow \beta$ in F , the dependency $\square \rightarrow (\alpha^+ - \alpha) \cap R_i$ can be shown to hold on R_i , and R_i violates BCNF.
 - We use above dependency to decompose R_i

BCNF and Dependency Preservation

- It is not always possible to get a BCNF decomposition that is dependency preserving
- $R = (J, K, L)$ $F = \{JK \rightarrow L, L \rightarrow K\}$
Two candidate keys = JK and JL
- R is not in BCNF
- Any decomposition of R will fail to preserve $JK \rightarrow L$

BCNF and Dependency Preservation

- BCNF decomposition does not always satisfy dependency preserving property.
- After BCNF decomposition if dependency is not preserved then we have to decide whether we want to remain in BCNF or rollback to 3NF.
- This process of rollback is called **denormalization**.

Multivalued Dependency

- Multivalued dependency occurs when two attributes in a table are independent of each other but, both depend on a third attribute.
- A multivalued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.
- MVD or multivalued dependency means that for a single value of attribute ' a ' multiple values of attribute ' b ' exist. We write it as, $a \twoheadrightarrow b$
- It is read as: **a is multi-valued dependent on b.**

Example - Multivalued Dependency

- A car manufacturing company that produces two of the colours in the market, i.e., red and yellow of each of their models, every year.

Carmodel	Manuf-month	Colour
2011	Jan	Yellow
2001	Feb	Red
3001	Mar	Yellow
3001	Apr	Red
4006	May	Yellow
4006	Jun	Red

Example - Multivalued Dependency

- COLOUR and MANUF_MONTH are dependent on CAR_MODEL, and they are independent of each other. Thus, we can call both of these columns multivalued. These are, as a result, dependent on CAR_MODEL.
- CARMODEL $\twoheadrightarrow$ MANUF_MONTH
- CARMODEL $\twoheadrightarrow$ COLOUR
- We can read this as "CARMODEL multidetermined MANUF_MONTH" and "CARMODEL multidetermined COLOUR".

Carmodel	Manuf-month	Colour
2011	Jan	Yellow
2001	Feb	Red
3001	Mar	Yellow
3001	Apr	Red
4006	May	Yellow
4006	Jun	Red

Why do we use MVD?

- When we want to test the relations or decide if these happen to be lawful under some arrangement of practical as well as multivalued dependencies.
- When we want to determine what limitations are there on the arrangement of the lawful relations. Thus, we will concern ourselves with just the relations that fulfil a given arrangement of practical as well as multivalued dependencies.

Conditions for MVD

- Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$.
- **The multivalued dependency** $\alpha \twoheadrightarrow \beta$ holds on R if in any legal relation $r(R)$, for all pairs for tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that:

$$t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$$

$$t_3[\beta] = t_1[\beta]$$

$$t_3[R - \beta] = t_2[R - \beta]$$

$$t_4[\beta] = t_2[\beta]$$

$$t_4[R - \beta] = t_1[R - \beta]$$

Conditions for MVD

- Tabular Representation of $\alpha \twoheadrightarrow \beta$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

Conditions for MVD

- Tabular Representation of $\alpha \twoheadrightarrow \beta$

A	B	C
Name	Project	Hobby
Ramya	MS	Reading
Ramya	Oracle	Music
Ramya	MS	Music
Ramya	Oracle	Reading

Conditions for MVD

- Tabular Representation of $\alpha \twoheadrightarrow \beta$

- $t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$

- $t_3[\beta] = t_1[\beta]$

- $t_3[R - ?] = t_2[R - ?]$

- $t_4[\beta] = t_2[\beta]$

- $t_4[R - ?] = t_1[R - ?]$

- **Condition-1:**

- $t_1[a] = t_2[a] = t_3[a] = t_4[a]$

- Finding from table, $t_1[a] = t_2[a] = t_3[a] = t_4[a] = \text{Ramya}$. Thus, condition 1 is satisfied here.

A	B	C
Name	Project	Hobby
Ramya	MS	Reading
Ramya	Oracle	Music
Ramya	MS	Music
Ramya	Oracle	Reading

Conditions for MVD

- Tabular Representation of $\alpha \twoheadrightarrow \beta$

- $t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$

- $t_3[\beta] = t_1[\beta]$

- $t_3[R - \alpha] = t_2[R - \alpha]$

- $t_4[\beta] = t_2[\beta]$

- $t_4[R - \alpha] = t_1[R - \alpha]$

A	B	C
Name	Project	Hobby
Ramya	MS	Reading
Ramya	Oracle	Music
Ramya	MS	Music
Ramya	Oracle	Reading

- **Condition-2:**

- $t_1[b] = t_3[b]$ & $t_2[b] = t_4[b]$. Finding from table, $t_1[b] = t_3[b] = \text{MS}$ & $t_2[b] = t_4[b] = \text{Oracle}$

- Thus, condition 2 is satisfied here.

Conditions for MVD

- Tabular Representation of $\alpha \twoheadrightarrow \beta$

- $t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$

- $t_3[\beta] = t_1[\beta]$

- $t_3[R - ?] = t_2[R - ?]$

- $t_4[\beta] = t_2[\beta]$

- $t_4[R - ?] = t_1[R - ?]$

- **Condition-3:**

- $t_1[c] = t_4[c]$ & $t_2[c] = t_3[c]$. Finding from table, $t_1[c] = t_4[c] = \text{Reading}$ & $t_2[c] = t_3[c] = \text{Music}$

- Thus, condition 3 is satisfied here.

A	B	C
Name	Project	Hobby
Ramya	MS	Reading
Ramya	Oracle	Music
Ramya	MS	Music
Ramya	Oracle	Reading

Conditions for MVD

A	B	C
Name	Project	Hobby
Ramya	MS	Reading
Ramya	Oracle	Music
Ramya	MS	Music
Ramya	Oracle	Reading

- All conditions are satisfied, thus
- $a \twoheadrightarrow b$ i.e. $Name \twoheadrightarrow Project$
- $a \twoheadrightarrow c$ i.e. $Name \twoheadrightarrow hobby$
- Thus, MVD exist in the given table and it stated like
- $Name \twoheadrightarrow Project, Name \twoheadrightarrow Hobby$.

Example - MVD

- Let R be a relation schema with a set of attributes that are partitioned into 3 nonempty subsets.

Y, Z, W

- We say that $Y \twoheadrightarrow Z$ (Y multidetermines Z) if and only if for all possible relations $r(R)$

$\langle y_1, z_1, w_1 \rangle \in r$ and $\langle y_2, z_2, w_2 \rangle \in r$ then

$\langle y_1, z_1, w_2 \rangle \in r$ and $\langle y_2, z_2, w_1 \rangle \in r$

- Note that since the behavior of Z and W are identical it follows that $Y \twoheadrightarrow Z$ if $Y \twoheadrightarrow W$

Example - MVD

course $\twoheadrightarrow$ *teacher*

course $\twoheadrightarrow$ *book*

- The above formal definition is supposed to formalize the notion that given a particular value of Y (*course*) it has associated with it a set of values of Z (*teacher*) and a set of values of W (*book*), and these two sets are in some sense independent of each other.
- Note:
 - If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$
 - Indeed we have (in above notation) $Z_1 = Z_2$
The claim follows.

Use of MVD

- We use multivalued dependencies in two ways:
 - To test relations to **determine** whether they are legal under a given set of functional and multivalued dependencies
 - To specify **constraints** on the set of legal relations. We shall thus concern ourselves *only* with relations that satisfy a given set of functional and multivalued dependencies.
- If a relation r fails to satisfy a given multivalued dependency, we can construct a relations r' that does satisfy the multivalued dependency by adding tuples to r .

Theory of MVD

- From the definition of multivalued dependency, we can derive the following rule:
 - If $\alpha \twoheadrightarrow \beta$, then $\alpha \sqsupset \beta$ i.e., every functional dependency is also a multivalued dependency
- The **closure** D^+ of D is the set of all functional and multivalued dependencies logically implied by D .
 - We can compute D^+ from D , using the formal definitions of functional dependencies and multivalued dependencies.
 - We can manage with such reasoning for very simple multivalued dependencies, which seem to be most common in practice
 - For complex dependencies, it is better to reason about sets of dependencies using a system of inference rules

Fourth Normal Form

- A relation schema R is in 4NF with respect to a set D of functional and multivalued dependencies if for all multivalued dependencies in D^+ of the form $\alpha \twoheadrightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following hold:
 - $\alpha \twoheadrightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$ or $\alpha \cup \beta = R$)
 - α is a superkey for schema R
- If a relation is in 4NF it is in BCNF

Fourth Normal Form

Restriction of MVD

- The restriction of D to R_i is the set D_i consisting of
 - All functional dependencies in D^+ that include only attributes of R_i
 - All multivalued dependencies of the form

$$\alpha \twoheadrightarrow (\beta \cap R_i)$$

where $\alpha \subseteq R_i$ and $\alpha \twoheadrightarrow \beta$ is in D^+

4NF Decomposition Algorithm

result := {*R*};

done := false;

compute D^+ ;

Let D_i denote the restriction of D^+ to R_i

while (**not** *done*)

if (there is a schema R_i in *result* that is not in 4NF) **then**

begin

 let $\alpha \twoheadrightarrow \beta$ be a nontrivial multivalued dependency that holds on R_i such that $\alpha \rightarrow R_i$ is not in D_i , and $\alpha \cap \beta = \emptyset$;

result := (*result* - R_i) $\cup$ (R_i - β) $\cup$ (α, β);

end

else *done* := true;

Note: each R_i is in 4NF, and decomposition is lossless-join

Fourth Normal Form - Example

- $R = (A, B, C, G, H, I)$

$F = \{ A \twoheadrightarrow B$

$B \twoheadrightarrow HI$

$CG \twoheadrightarrow H \}$

- R is not in 4NF since $A \twoheadrightarrow B$ and A is not a superkey for R

Fourth Normal Form - Example

- Decomposition

a) $R_1 = (A, B)$ $(R_1 \text{ is in } 4NF)$

b) $R_2 = (A, C, G, H, I)$ $(R_2 \text{ is not in } 4NF)$

c) $R_3 = (C, G, H)$ $(R_3 \text{ is in } 4NF)$

d) $R_4 = (A, C, G, I)$ $(R_4 \text{ is not in } 4NF)$

- Since $A \twoheadrightarrow B$ and $B \twoheadrightarrow HI$, $A \twoheadrightarrow HI$, $A \twoheadrightarrow I$

e) $R_5 = (A, I)$ $(R_5 \text{ is in } 4NF)$

f) $R_6 = (A, C, G)$ $(R_6 \text{ is in } 4NF)$

Join Dependencies

- Join dependency or JD is a constraint that is similar to FD (functional dependency) or MVD (multivalued dependency).
- JD is satisfied only when the concerned relation is a join of a specific number of projections. Thus, such a type of constraint is known as a **join dependency**.

Join Dependencies

- Whenever we can recreate a table by simply joining various tables where each of these tables consists of a subset of the table's attribute, then this table is known as a **Join Dependency**.
- Thus, it is like a generalization of MVD. We can relate the JD to 5NF.
- Herein, a relation can be in 5NF only when it's already in the 4NF.
- Remember that it cannot be further decomposed.

Join Dependencies - Example

Student Name	Student Skills	Student Job
Tom	Marketing	001
Bavani	PR	002
Pricy	Design	003

- We can decompose the table given above into these three tables given below. And thus, it is not in the Fifth Normal Form.

Student Name	Student Skills
Tom	Marketing
Bavani	PR
Pricy	Design

Student Name	Student Job
Tom	001
Bavani	002
Pricy	003

Student Skills	Student Job
Marketing	001
PR	002
Design	003

Join Dependencies - Example

Student Name	Student Skills
Tom	Marketing
Bavani	PR
Pricy	Design

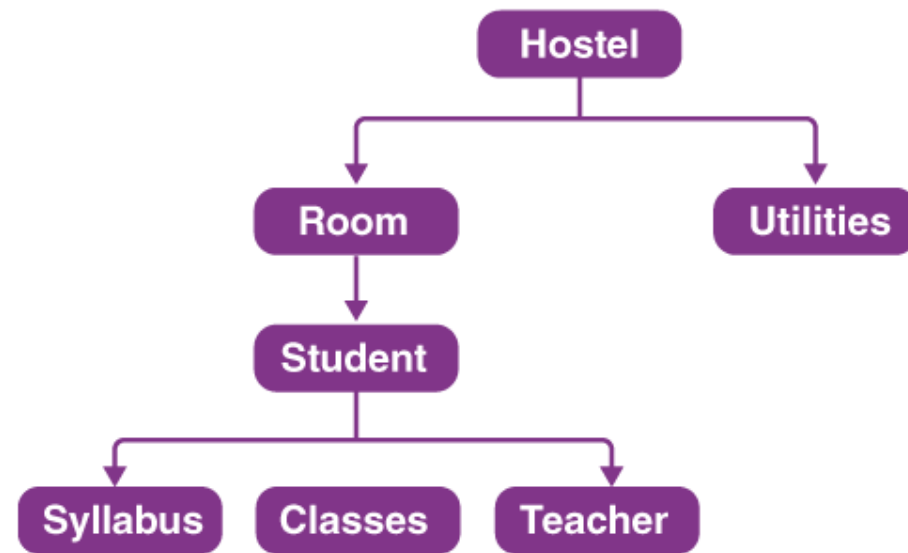
Student Name	Student Job
Tom	001
Bavani	002
Pricy	003

Student Skills	Student Job
Marketing	001
PR	002
Design	003

- Our Join Dependency would be:
- $\{(Stu_Name, Stu_Skills), (Stu_Name, Stu_Job), (Stu_Skills, Stu_Job)\}$
- The relations given above have join dependency. Thus, they do not happen to be in 5NF. It means that the join relation of the three relations given above is equal to the very original relation $\langle Student \rangle$.

Join Dependencies - Example

- Let us consider some special classes of join dependencies that help us in capturing data dependencies that are present in a data structure that is hierarchical in nature.



Join Dependencies - Example

- This hierarchical organisation informs the reader about the rooms, and the students currently living in the room depend only on the hostel but not the utilities present in that hostel. Since hostels have multiple rooms, FDs are NOT adequate when we want to describe the data dependency among hostels and rooms or utilities.
- In such a case, the multivalued dependencies,
- **Hostel ->-> room** or
- **Hostel ->-> utilities** hold

Join Dependencies - Example

- The first-order hierarchical decomposition, one would be enabled to represent data dependencies that are present in a hierarchical data structure in a natural way.
- Thus, one can store the hostel database as a lossless join of the following:
 - Hostel_utility (hostel, utilities),
 - Hostel_room (hostel, room, student, syllabus, classes, teacher)

Fifth Normal Form

- A relation R is in Fifth Normal Form (5NF) if and only if the following conditions are satisfied simultaneously:
 1. **R is already in 4NF.**
 2. **It cannot be further non-loss decomposed.**
- 5NF is of little practical use to the database designer, but it is of interest from a theoretical point of view and a discussion of it is included here to complete the picture of the further normal forms.

Fifth Normal Form

- In all of the further normal forms discussed so far, no loss decomposition was achieved by the decomposing of a single table into two separate tables.
- No loss decomposition is possible because of the availability of the join operator as part of the relational model.
- In considering 5NF, consideration must be given to tables where this non-loss decomposition can only be achieved by decomposition into three or more separate tables.
- Such decomposition is not always possible as is shown by the following example. Consider the table below.
- agent_company_product (Agent, Company, Product)

Fifth Normal Form

- This table lists agents, the companies they work for and the products they sell for those companies.
- 'The agents do not necessarily sell all the products supplied by the companies they do business with.
- An example of this table might be:

Agent	Company	Product
Karthik	ABC	Nut
Karthik	ABC	Bolt
Karthik	XYZ	Nut
Karthik	XYZ	Bolt
Ramya	ABC	Nut

Fifth Normal Form

- The table is necessary in order to show all the information required.
- Karthik, for example, sells ABC's and XYZ's - Nuts and Bolts. Ramya sells ABC's Nuts alone.

Agent	Company	Product
Karthik	ABC	Nut
Karthik	ABC	Bolt
Karthik	XYZ	Nut
Karthik	XYZ	Bolt
Ramya	ABC	Nut

Fifth Normal Form

- The table is in 4NF because it contains no multi-valued dependency.
- It does, however, contain an element of redundancy in that it records the fact that
- Karthik is an agent for ABC twice.

Agent	Company	Product
Karthik	ABC	Nut
Karthik	ABC	Bolt
Karthik	XYZ	Nut
Karthik	XYZ	Bolt
Ramya	ABC	Nut

Fifth Normal Form

- But there is no way of eliminating this redundancy without losing information. Suppose that the table is decomposed into its two projections, R1 and R2.

Agent	Company
Karthik	ABC
Karthik	XYZ
Ramya	ABC

Company	Product
ABC	Nut
ABC	Bolt
XYZ	Nut
XYZ	Bolt

Fifth Normal Form

- The redundancy has been eliminated, but the information about which companies make which products and which of these products they supply to which agents has been lost. The natural join of these projections over the 'agent' columns is:

Agent	Company	Product
Karthik	ABC	Nut
Karthik	ABC	Bolt
Karthik	XYZ	Nut
Karthik	XYZ	Bolt
Ramya	ABC	Nut
Ramya	ABC	Bolt

Fifth Normal Form

- The table resulting from this join is spurious, since the asterisked row of the table contains incorrect information.

Agent	Company	Product
Karthik	ABC	Nut
Karthik	ABC	Bolt
Karthik	XYZ	Nut
Karthik	XYZ	Bolt
Ramya	ABC	Nut
Ramya	ABC	Bolt

Fifth Normal Form

- Now suppose that the original table were to be decomposed into three tables, the two projections, R1, R2 and R3 which is .

Agent	Company
Karthik	ABC
Karthik	XYZ
Ramya	ABC

Agent	Product
Karthik	Nut
Karthik	Bolt
Ramya	Nut

Company	Product
ABC	Nut
ABC	Bolt
XYZ	Nut
XYZ	Bolt

Fifth Normal Form

Agent	Company
Karthik	ABC
Karthik	XYZ
Ramya	ABC

Company	Product
ABC	Nut
ABC	Bolt
XYZ	Nut
XYZ	Bolt

- Then of this result with R2 over the 'Company' and 'Product name' column, the following table is obtained:..

Agent	Company	Product
Karthik	ABC	Nut
Karthik	ABC	Bolt
Karthik	XYZ	Nut
Karthik	XYZ	Bolt
Ramya	ABC	Nut
Ramya	ABC	Bolt

Fifth Normal Form

Agent	Product
Karthik	Nut
Karthik	Bolt
Ramya	Nut

- If a join is taken of all three projections, first of R1 and R3 with the (spurious) result shown, and then of this result with R2 over the 'Company' and 'Product name' column, the following table is obtained:.

Agent	Company	Product
Karthik	ABC	Nut
Karthik	ABC	Bolt
Karthik	XYZ	Nut
Karthik	XYZ	Bolt
Ramya	ABC	Nut
Ramya	ABC	Bolt

Agent	Company	Product
Karthik	ABC	Nut
Karthik	ABC	Bolt
Karthik	XYZ	Nut
Karthik	XYZ	Bolt
Ramya	ABC	Nut

Fifth Normal Form

- This is a correct re-composition of the original table and no loss decomposition into the three projections was achieved.
- Again, the order in which the joins are performed does not affect the final result.
- The original table, therefore, violated 5NF simply because it was non-loss decomposable into its three projections.

Fifth Normal Form

- If a table is non-loss decomposable as in the second case, it is said to be in violation of 5NF.
- The difference, of course, lay in certain semantic properties of the information being represented.
- These properties were not understandable simply by looking at the table, but had to be supplemented by further information about the relationship between products, agents and companies.

Fifth Normal Form

- Detecting that a table violates 5 NF is very difficult in practice and for this reason this normal form has little if any practical application. The theoretical concept of fifth normal form is discussed in the following paragraphs.
- Suppose that the statement, 'The agent-company-product' table is equal to the join of its three projections is to hold true, this is another way of saying that it Can be non-loss decomposed into its three projections and is equivalent to saying.
- IF the tuple 'agent X, company Y' appears in R1
- AND the tuple 'agent X, product Z' appears in R3
- AND the tuple 'company Y, product Z' appears in R2

Fifth Normal Form

- The rule is referred to as a Join. Dependency, because it holds good only if a table can be reconstituted without loss of information from the join of certain specified projections it.
- The notation used for a join dependency on table T is:
- $\star(X, Y, Z)$ Where $X, Y \dots Z$ are projections of T.
- Table T is said to satisfy the above join dependency, if it is equal to the join of the projections X, Y, Z .
- Thus, the second example given of the table 'agent_company product' can be said to satisfy the join dependency: $\star(R1, R2, R3)$

Fifth Normal Form

- A table T is in fifth normal form if every join dependency in T is a consequence only of the candidate keys of T.
- The second version of the table 'agent_company product' illustrated earlier' violated
- 5NF, because the join dependency $\ast(\text{agent}, \text{company}, \text{product_name})$ was not a consequence only of the primary key for the table, but also a consequence of the tuple formation rule which was given earlier,
- It can be shown that if a table is in 5NF, then, because join dependencies are the 'ultimate' form of dependency, it must also be in 4NF and thus confirm to all the further normal forms. The problem with this is that detecting join dependencies is, in practice, very difficult. For this reason, 5NF is largely of academic interest.

Fifth Normal Form

- A table T is in fifth normal form if every join dependency in T is a consequence only of the candidate keys of T.
- The second version of the table 'agent_company product' illustrated earlier' violated
- 5NF, because the join dependency $\ast(\text{agent}, \text{company}, \text{product_name})$ was not a consequence only of the primary key for the table, but also a consequence of the tuple formation rule which was given earlier,
- In the first, example of 'agent_company Product' there was no application of this rule, hence no join dependency other than that on the primary key. Thus, the table was in 5NF.
- It can be shown that if a table is in 5NF, then, because join dependencies are the 'ultimate' form of dependency, it must also be in 4NF and thus confirm to all the further normal forms. The problem with this is that

THANK YOU

Module 3

BCSE302L – Database Systems